

End-to-End Secure Real-Time Chat Application

Complete End-to-End Development Instructions

Course	Secure Software Design
Instructor	Dr. Zubair Ahmad
Team Members	M. Abdullah Mahsud (2023346) M. Zamad Safi (2023552) Hassan Khan (2023243)
Department	Cyber Security
Date	February 27, 2026
Version	1.0 — Final Submission Brief
Classification	CONFIDENTIAL — Internal Use Only

1. Project Overview

This document serves as the complete development brief for your Secure Real-Time Chat Application. It is structured the same way a client would hand over a project to a development team — each section tells you what to build, why it matters, and what success looks like. Follow the phases in order without skipping ahead.

1.1 What You Are Building

A fully functional, enterprise-grade chat application where users can register, log in, and exchange real-time messages. Every component must be built with security as the first priority, not an afterthought. The application will be tested against real-world attack tools before submission.

1.2 Technology Stack

Component	Technology & Purpose
User Interface	Next.js — The web frontend users interact with in their browser
Application Server	FastAPI (Python) — Handles all business logic, authentication, and API requests
Real-Time Messaging	WebSockets over WSS — Delivers messages instantly without page refresh
Database	PostgreSQL — Stores all user data and messages with encryption
Containerization	Docker — Packages the entire application so it runs identically everywhere
Automation Pipeline	GitHub Actions — Automatically tests and scans the code on every update
Security Testing	Nessus + OWASP ZAP — Simulates real attacks to find vulnerabilities

1.3 Master Timeline

Week	Work to Complete
Week 1	Environment setup, project structure, and the entire backend authentication system
Week 2	Database schema, message encryption, and real-time WebSocket server
Week 3	Frontend user interface, Docker containers, and CI/CD pipeline
Week 4	Security scanning with Nessus and ZAP, fixing vulnerabilities, writing the final report

Phase

1

Environment Setup

Duration: Days 1–2 — Nothing gets skipped here. A bad setup causes problems in every phase after it.

2.1 Software to Install on Your Machine

Install the following tools before writing a single line of application code. Verify each one works by checking its version number in the terminal after installation.

- Python 3.11 or higher — Download from python.org. This is the language your backend server is written in.
- Node.js version 18 or higher — Download the LTS version from nodejs.org. This is needed to run the frontend.
- Docker Desktop — Download from docker.com. Available for both Windows and Mac. This lets you run your app in containers.
- Git — For version control. Your entire project must be tracked in a Git repository.
- Visual Studio Code — Your code editor. After installing, add these three extensions: Python, ESLint, and Docker.
- Postman — A tool for testing your API endpoints. You will use this constantly during backend development.

2.2 Project Folder Structure

Create the following folder structure manually. Every team member must use exactly this structure. This is non-negotiable because the Docker configuration depends on these exact paths.

■ Required Folder Structure

secure-chat/

■■■■ backend/ ← All Python/FastAPI server code lives here

■■■■ frontend/ ← All Next.js interface code lives here

■■■■ docker/ ← Dockerfile configurations for each service

■■■■ .github/

■ ■■■■ workflows/ ← GitHub Actions CI/CD pipeline files

■■■■ security/ ← Nessus and ZAP configuration files

2.3 Environment Variables File

Create a file named `.env` inside your backend folder. This file stores all secret values such as your database password and JWT signing key. This file must NEVER be committed to Git. Add `.env` to your `.gitignore` file immediately.

■ Critical Security Rule

The `.env` file contains your application's secrets. If this file is ever pushed to GitHub, your application is compromised.

Action required: Add `.env` to `.gitignore` before your very first Git commit. Verify it is ignored before proceeding.

2.4 Phase 1 Completion Checklist

1. Python, Node.js, Docker, and Git all respond with a version number when typed in the terminal.
2. The full folder structure exists and all team members have cloned the repository.
3. The `.env` file exists and is confirmed to be listed in `.gitignore`.
4. Postman is installed and can reach localhost when Docker is running.

Phase

2

Backend Development — FastAPI Server

Duration: Days 3–7 — This is the most complex phase. Addresses OWASP A01, A02, and A07.

■ Security Objective

This phase addresses three of the OWASP Top 10 vulnerabilities:

A01 — Broken Access Control: Only the right users access the right data.

A02 — Cryptographic Failures: Passwords are hashed, tokens are signed, data is encrypted.

A07 — Identification and Authentication Failures: Login is rate-limited, JWTs expire.

3.1 What the Backend Must Do

The FastAPI backend is the brain of your application. It must perform all of the following functions. Each one is a discrete deliverable that can be tested independently using Postman.

3.1.1 User Registration

When a new user submits their email and password, the server must validate the input, check that the email does not already exist in the database, hash the password using the bcrypt algorithm, save the hashed password to the database, and return a success response. Under no circumstances should the plain-text password ever be stored or logged anywhere.

3.1.2 User Login with Rate Limiting

When a user attempts to log in, the server must compare the submitted password against the stored hash, reject invalid attempts with a generic error message that does not reveal whether the email or password was wrong, enforce a limit of five login attempts per minute per IP address to prevent brute-force attacks, and issue a signed JWT token on success. The token must have an expiry time of 30 minutes.

■ Why Generic Error Messages?

Saying 'password incorrect' confirms the email exists — which is information an attacker can use.

The correct message is: 'Invalid credentials' — this reveals nothing about which field is wrong.

3.1.3 JWT Token Verification

Every protected endpoint must check the JWT token in the request header before proceeding. If the token is missing, expired, or tampered with, the server must reject the request with a 401 Unauthorized response. This token check must be implemented as a reusable middleware function, not copied into each endpoint individually.

3.1.4 Role-Based Access Control

The system must support two user roles: regular user and administrator. A regular user can only access their own messages and profile. An administrator can view all users, all rooms, and all audit logs. Role checks must be enforced on the server side. A user claiming to be an admin in their request must be rejected if their token does not confirm it.

3.1.5 Input Validation on Every Endpoint

Every field accepted by every endpoint must be validated before it is used. Email fields must be verified as real email format. Password fields must meet a minimum strength requirement of eight characters including at least one uppercase letter and one number. Message content must be stripped of leading and trailing whitespace and rejected if blank or longer than 2,000 characters. This validation is your primary defense against injection attacks.

3.1.6 Security Headers Middleware

The server must attach the following HTTP security headers to every single response it sends. These headers instruct browsers to enforce strict security policies and are verified by OWASP ZAP during testing.

Header Name	What It Does
X-Content-Type-Options	Prevents browsers from guessing the content type, blocking MIME-sniffing attacks
X-Frame-Options: DENY	Prevents your application from being embedded in an iframe, blocking clickjacking
X-XSS-Protection	Enables the browser's built-in cross-site scripting filter
Strict-Transport-Security	Forces all future connections to use HTTPS only
Content-Security-Policy	Restricts which scripts, styles, and resources the browser is allowed to load

3.2 Phase 2 Completion Checklist

1. Registration endpoint creates a user with a bcrypt-hashed password. Confirm by checking the database directly — the stored value must look like a long hash string, not the original password.
2. Login endpoint returns a JWT token on success and rejects invalid credentials with a generic message.
3. A sixth login attempt within one minute is rejected with a 429 Too Many Requests response.
4. Accessing any protected endpoint without a token returns 401 Unauthorized.
5. A regular user attempting to access an admin-only endpoint is rejected with 403 Forbidden.
6. All five security headers are present in every response. Verify using Postman by inspecting the response headers tab.

Phase

3

Database Design — PostgreSQL

Duration: Days 8–9 — Addresses OWASP A03 (Injection) and A09 (Security Logging Failures).

4.1 Database Security Principles

The database is the most sensitive component of the system. Every design decision must protect the data stored in it. The following rules are mandatory and will be checked during the security testing phase.

- Create a dedicated database user for the application. This user must have only the permissions it needs and nothing more. Never connect your application to the database using a superuser or admin account.
- Never construct database queries by joining strings together. Use parameterized queries or an ORM at all times. This is the primary defense against SQL injection.
- Store message content in encrypted form. The database should hold ciphertext, not plaintext. Even if an attacker gains direct database access, they must not be able to read messages.
- Every significant action in the system must create a record in the audit log table. This is not optional.

4.2 Required Tables

Your database must contain exactly the following four tables. Do not rename them, as the backend code will reference them by these names.

Table Name	Purpose and Required Columns
users	Stores registered accounts. Must include: unique ID, email address (unique), hashed password, user role, active status flag, and registration timestamp.
rooms	Stores chat rooms. Must include: unique ID, room name, the ID of the user who created it, and creation timestamp.
messages	Stores all chat messages. Must include: unique ID, room ID (foreign key), sender ID (foreign key), encrypted message content stored as binary, and timestamp. Message content column must store ciphertext, never plaintext.
audit_logs	Records all significant system events. Must include: unique ID, user ID (nullable for failed logins), action label such as LOGIN_SUCCESS, the IP address of the request, optional metadata in JSON format, and timestamp.

4.3 Events That Must Be Logged

The following events must automatically create a record in the audit_logs table. This list is the minimum requirement. You may log additional events.

- Successful user login

- Failed login attempt (including the IP address)
- User registration
- User logout
- Every message sent
- Every admin action such as deleting a user or viewing all messages

4.4 Phase 3 Completion Checklist

1. All four tables exist with the correct columns. Verify by connecting to the database and listing them.
2. A test login creates a record in audit_logs. Verify by querying the table immediately after.
3. A test message is stored as unreadable binary data in the messages table. Verify by reading the content column directly — it must not be readable text.
4. Attempting an SQL injection in any input field produces an error or is silently ignored — it must never return unexpected data.

Phase

4

Real-Time Messaging — WebSocket Server

Duration: Days 10–12 — The core chat feature. All connections must be authenticated and all messages validated.

5.1 How the WebSocket System Must Work

WebSockets allow the server and client to maintain an open, two-way connection. Unlike regular HTTP requests, this connection stays open so the server can push new messages to the user instantly. Your implementation must follow this exact flow:

1. A user opens the chat page in their browser. The browser attempts to open a WebSocket connection to the server and sends the JWT token as part of the connection request.
2. The server receives the connection attempt and immediately verifies the JWT token. If the token is missing, expired, or invalid, the server closes the connection before accepting it. The user sees a connection error and is redirected to the login page.
3. If the token is valid, the server accepts the connection and adds this user to the active connections list for their chat room.
4. When the user types a message and presses send, the browser sends the text to the server through the open WebSocket connection.
5. The server receives the message, validates that it is not empty and not longer than 2,000 characters, encrypts it, saves the encrypted version to the database, creates an audit log entry, and then broadcasts the original text to all other active users in the same room.
6. When the user closes the chat window or navigates away, the server detects the disconnection and removes them from the active connections list.

■ Security Requirement: WSS Not WS

You must use WSS (WebSocket Secure), which is WebSocket over TLS — exactly like HTTPS but for real-time connections.

Plain WS (unencrypted WebSocket) transmits messages in cleartext and will be flagged as a critical vulnerability by Nessus.

In development: use self-signed certificates. In production: use a proper certificate from Let's Encrypt.

5.2 Connection Management Requirements

Your server must maintain a record of which users are currently connected to which rooms. The following rules apply:

- One user can be connected to one room at a time in the basic implementation.
- When a user disconnects unexpectedly (browser crash, lost internet), the server must detect this and remove them from the active list.
- When a user joins a room, all other users in that room should receive a notification message.
- When a user leaves a room, all other users should be notified.

5.3 Phase 4 Completion Checklist

1. Opening the chat page without a valid JWT token is automatically rejected. The connection must never be accepted.
2. Two browser windows logged in as different users can exchange messages in real time without refreshing.
3. The messages table in the database shows encrypted binary data after a test conversation, not readable text.
4. Sending an empty message or a message over 2,000 characters is silently dropped by the server without causing an error.
5. Closing one browser window does not cause the other user's chat to break.

Phase

5

Frontend Development — Next.js Interface

Duration: Days 13–17 — Users interact with this layer. Security must be built in, not added on top.

6.1 Required Pages

Page	What It Must Do
Registration Page	Collects email and password. Validates both fields client-side before submitting. Shows clear error messages for weak passwords. Sends data to the registration endpoint on the backend.
Login Page	Collects email and password. Submits to the login endpoint. On success, stores the JWT token in an httpOnly cookie. Redirects to the chat page. On failure, shows a generic error without revealing which field was wrong.
Chat Page	Loads only if the user is authenticated. Opens the WebSocket connection on page load. Displays incoming messages in real time. Sends messages when the user presses enter. Shows a list of online users.
Admin Dashboard	Accessible only to users with the admin role. Shows a list of all registered users. Shows recent audit log entries. Allows an admin to deactivate a user account.

6.2 Token Storage Rule

■ Mandatory Rule — How to Store the JWT Token

CORRECT: Store the JWT token in an httpOnly cookie with the Secure and SameSite=Strict flags.

FORBIDDEN: Never store the JWT token in localStorage or sessionStorage.

Reason: JavaScript code running in the browser cannot read httpOnly cookies, which means an XSS attack cannot steal the token. localStorage and sessionStorage are fully readable by any script on the page, making them completely unsafe for authentication tokens.

6.3 Security Headers in Next.js

The frontend must configure Next.js to send all the same security headers described in Phase 2 on every page response. This is done once in the Next.js configuration file and applies automatically to all pages. This is not optional — OWASP ZAP will scan the frontend and flag missing headers as vulnerabilities.

6.4 Input Handling

- Validate input client-side to improve user experience, but always validate again on the server side. Client-side validation is not a security control.

- Never insert user-provided text into the page using methods that interpret it as HTML. Always use the safe text-insertion methods provided by React.
- Limit message length on the input element itself to 2,000 characters as a first line of defense.

6.5 Phase 5 Completion Checklist

1. Navigating directly to the chat page URL without being logged in redirects to the login page.
2. After login, the browser's developer tools Application tab shows the token in Cookies, not in Local Storage.
3. Attempting to type an HTML script tag into the message box and send it displays the literal text on screen, not as an executed script.
4. The admin dashboard is not accessible when logged in as a regular user — it must return a 403 error.
5. All five security headers are present in the page response. Verify in the browser's Network tab.

Phase

6

Containerization — Docker & CI/CD Pipeline

Duration: Days 18–20 — Package the entire application for consistent, secure deployment.

7.1 Why Docker Is Required

Docker ensures that the application behaves identically on every machine and in every environment. It eliminates the problem of code working on one developer's laptop but failing on another's. For this project, Docker also provides an isolation boundary — each service runs in its own container and can only communicate through defined channels.

7.2 Docker Configuration Requirements

- Do not run any container as the root user. Create a dedicated non-root user inside each Dockerfile and switch to that user before starting the application.
- Do not include .env files, secret keys, API keys, or passwords inside any Docker image. Pass secrets through environment variables at runtime.
- Do not include development tools, debuggers, or test dependencies in the production Docker image. Use multi-stage builds if needed.
- Use specific version tags for base images. Never use the 'latest' tag, as it changes without warning and can introduce vulnerabilities.

7.3 CI/CD Pipeline Requirements

The GitHub Actions pipeline must automatically run every time code is pushed to the repository. If any step fails, the pipeline must stop and report the failure.

1. Run all automated tests for the backend. If any test fails, the pipeline stops here.
2. Scan all Python packages listed in requirements.txt for known vulnerabilities using an automated tool. Flag any high-severity findings.
3. Run a static code analysis check to catch common security mistakes in the code.
4. Build the Docker images to confirm they build successfully.
5. Only after all the above steps pass, deploy the updated application.

7.4 Phase 6 Completion Checklist

1. Running a single docker-compose up command starts the entire application including the database.
2. Inspecting the running containers confirms that none of them are running as the root user.
3. Pushing code to the main branch triggers the GitHub Actions pipeline automatically. Confirm by checking the Actions tab on GitHub.
4. Introducing a deliberately vulnerable Python package triggers a pipeline failure.

Phase

7

Security Testing — Nessus & OWASP ZAP

Duration: Days 21–25 — The most academically important phase. Demonstrates your course learning outcomes.

■ Purpose of This Phase

This phase demonstrates that you understand not just how to build software but how to attack it and defend it. You are expected to find vulnerabilities, document them, fix them, and prove they are fixed by running the scan again.

A perfect first scan with zero findings is not the goal — a thorough analysis and evidence of remediation is.

8.1 Nessus Scanning Instructions

8.1.1 Setup

- Download and install Nessus Essentials from the Tenable website. Register with your university email to receive a free activation code.
- Ensure your application is fully running in Docker before starting any scan.
- Confirm that Nessus and your application can reach each other on the network — both must be on the same machine or local network.

8.1.2 Running the Scan

- Open the Nessus web interface at <https://localhost:8834> in your browser.
- Create a new scan. Select the Basic Network Scan template as your starting point.
- Enter the IP address and port of your deployed application as the target.
- Launch the scan and wait for it to complete. The first scan may take 15 to 30 minutes.
- Export the full scan report in PDF format. This report is a required submission deliverable.

8.1.3 After the Scan

- Review every finding. Categorize each one as Critical, High, Medium, Low, or Informational.
- For every Critical and High finding, research the vulnerability, implement a fix, and run the scan again to confirm it is resolved.
- Document this entire process in your final report: what was found, what you changed, and the result of the re-scan.

8.2 OWASP ZAP Scanning Instructions

OWASP ZAP is a web application scanner that tests specifically for web vulnerabilities including those in the OWASP Top 10.

- Download and install OWASP ZAP from [zaproxy.org](https://www.zaproxy.org/).
- Open ZAP and enter the URL of your running application in the Quick Start tab.

- Run the Automated Scan. ZAP will first crawl your application to discover all pages and endpoints, then actively probe them for vulnerabilities.
- Wait for the scan to complete. Review the Alerts tab, which categorizes findings by severity.

Vulnerability Category	What ZAP Is Looking For
SQL Injection	Whether your database queries can be manipulated through user input
Cross-Site Scripting (XSS)	Whether an attacker can inject scripts into pages viewed by other users
Missing Security Headers	Whether all five required headers are present on every response
CSRF Vulnerabilities	Whether forms can be submitted on behalf of a logged-in user from another site
Sensitive Data Exposure	Whether error messages reveal stack traces, file paths, or database details
Insecure Cookie Flags	Whether session cookies have the Secure and HttpOnly flags set

8.3 Final Security Report Requirements

Your final report must contain the following sections in this order. This report is submitted along with the working application.

1. Executive Summary — One page. Non-technical description of what was built, what was tested, and the overall security posture of the application.
2. Architecture Overview — A diagram showing all components and how they connect, with annotations explaining each security control.
3. OWASP Top 10 Mapping — A table showing which items from the OWASP Top 10 are addressed by your application and how.
4. Nessus Findings — Include the exported Nessus report. For each Critical and High finding, describe the vulnerability, the fix you applied, and the evidence of the fix working.
5. ZAP Findings — Include screenshots of the ZAP alerts panel before and after fixes. For each High finding, describe the vulnerability and remediation.
6. Lessons Learned — What vulnerabilities surprised you? What would you do differently if starting over?

8.4 Phase 7 Completion Checklist

1. Nessus scan report has been exported as PDF and is ready for submission.
2. All Critical and High Nessus findings have been remediated and confirmed by a re-scan.
3. ZAP shows zero High severity alerts in the final scan. Medium alerts are documented with explanations.
4. The final security report covers all six required sections listed above.
5. The working application and all source code are committed to the Git repository with a clean, readable commit history.

9. Full Deliverables Summary

The table below lists every item that must be submitted. Use this as your final pre-submission checklist.

Deliverable	Responsible	Due	Priority
Working application running in Docker	All members	Week 3	CRITICAL
Git repository with full commit history	All members	Ongoing	CRITICAL
FastAPI backend with all security controls	Backend team	Week 1-2	HIGH
PostgreSQL schema with audit logging	Backend team	Week 2	HIGH
WebSocket server with authentication	Backend team	Week 2	HIGH
Next.js frontend with all four pages	Frontend team	Week 3	HIGH
Docker Compose configuration	All members	Week 3	HIGH
GitHub Actions CI/CD pipeline	All members	Week 3	MEDIUM
Nessus scan report (PDF export)	Security team	Week 4	CRITICAL
OWASP ZAP before and after screenshots	Security team	Week 4	CRITICAL
Final security report (all six sections)	All members	Week 4	CRITICAL

10. Common Mistakes to Avoid

These are the most frequently made mistakes on projects of this type. Review this list at the start of each phase.

■ Mistake 1 — Leaving Security for the End

Security controls must be built in from the start, not added on top of a finished application.

If you build registration without hashing passwords and plan to add hashing later, you will find that it affects many other parts of the system.

Rule: Every feature must be built with its security requirement from day one.

■ Mistake 2 — Storing Secrets in Git

Pushing your .env file, database password, or JWT secret key to GitHub is a critical security failure.

Even if you delete the file later, it remains in the Git history and can be recovered.

Rule: Add .env to .gitignore before your very first commit. Never hardcode secrets in your source code.

■ Mistake 3 — Using Plain WS Instead of WSS

If your WebSocket URL starts with ws:// instead of wss://, all messages are transmitted in plain text.

Nessus and ZAP will both flag this as a critical vulnerability.

Rule: Always use wss:// for WebSocket connections, exactly as HTTPS is required over HTTP.

■ Mistake 4 — Storing Tokens in localStorage

localStorage is fully readable by JavaScript running on the page. If your site has even one XSS vulnerability, an attacker can steal every token.

Rule: Always store JWT tokens in httpOnly cookies. This is not a preference — it is a security requirement for this project.

■ Mistake 5 — Running Containers as Root

If a container is compromised while running as root, the attacker has root-level access to the host machine.

Docker Nessus scans specifically check for this.

Rule: Every Dockerfile must create a non-root user and switch to that user before the CMD or ENTRYPOINT instruction.

■ Mistake 6 — Only Running Nessus Once

The scan-fix-rescan cycle is what demonstrates real understanding of security.

A single scan report with unresolved findings will not demonstrate the remediation process.

Rule: Run Nessus at least twice — once to find issues and once to confirm fixes. Include both reports.

Good luck with your project.

Follow the phases in order, test early, and never skip